مكتب التكوين المهني و إنعاش الشغل
Office de la Formation Professionnelle
et de la Promotion du Travail

# Compte-rendu

## DEVOIR N° 1

Prénom : AYMEN
Nom : EL YAAKOUBI
Groupe : DEV106

Année scolaire : 2025/2026

# INTRODUCTION:

Dans ce document, je présente les réponses du devoir de Python réalisées à l'aide de l'environnement Py.Charm. Ce travail m'a permis de mettre en pratique les notions fondamentales du langage Python, telles que les structures de contrôle, les fonctions et la manipulation des données. Les solutions proposées sont organisées de manière claire afin de faciliter la compréhension et la lecture.

Exercice 1:

Question 1:

```python
from abc import ABC, abstractmethod
import json
class Animal(ABC):  5 usages
    def __init__(self, code, nom_scientifique, espece, nombre_de_pieds, pays_origine):
        self.code = code
        self.nom_scientifique = nom_scientifique
        self.espece = espece
        self.nombre_de_pieds = nombre_de_pieds
        self.pays_origine = pays_origine
```

Figure 1: Classe abstraite Animal.

Cette photo représente une classe abstraite Python (Animal) utilisant le module abc. Elle définit un constructeur (init) qui initialise cinq attributs d'instance (état de l'objet).

Question 2:

```python
class Animal(ABC):  5 usages
    def get_code(self):
        return self.code

    def get_nom_scientifique(self):
        return self.nom_scientifique

    def get_espece(self):
        return self.espece

    def get_nombre_de_pieds(self):
        return self.nombre_de_pieds

    def get_pays_origine(self):
        return self.pays_origine

    def set_nom_scientifique(self, nom):  1 usage (1 dynamic)
        self.nom_scientifique = nom

    def set_espece(self, espece):  1 usage (1 dynamic)
        self.espece = espece

    def set_nombre_de_pieds(self, nb):  1 usage (1 dynamic)
        self.nombre_de_pieds = nb

    def set_pays_origine(self, pays):  1 usage (1 dynamic)
        self.pays_origine = pays
```

Figure 2: les sélecteurs et les mutateurs.

Cette photo représente des Accesseurs (Getters) et des Mutateurs (Setters). Ils servent à implémenter le principe d'encapsulation en POO (Programmation Orientée Objet).

Question 3:

```
def afficher_info(self):  6 usages (2 dynamic)
    print(f"Code: {self.code}")
    print(f"Nom scientifique: {self.nom_scientifique}")
    print(f"Espèce: {self.espece}")
    print(f"Nombre de pieds: {self.nombre_de_pieds}")
    print(f"Pays d'origine: {self.pays_origine}")

@abstractmethod
def se_deplacer(self):
    pass
```

Figure 3: une méthode Afficher_info().

Cette photo représente la méthode afficher_info traite de la gestion de l'état interne d'un objet en accédant à ses attributs, une pratique souvent couplée aux getters et setters pour protéger l'intégrité des données.

Question 4, 5, 6, 7:

```
class Vache(Animal):  1 usage
    def se_deplacer(self):
        print(f"La vache marche sur {self.nombre_de_pieds} pattes.")

    def afficher_info(self):  2 usages (2 dynamic)
        print("=== Vache ===")
        super().afficher_info()
```

```
class Chien(Animal):  2 usages
    def se_deplacer(self):
        print(f"Le chien marche sur {self.nombre_de_pieds} pattes.")

    def afficher_info(self):  2 usages (2 dynamic)
        print("=== Chien ===")
        super().afficher_info()

class Chat(Animal):  2 usages
    def se_deplacer(self):
        print(f"Le chat marche sur {self.nombre_de_pieds} pattes.")

    def afficher_info(self):  2 usages (2 dynamic)
        print("=== Chat ===")
        super().afficher_info()

class Poulet(Animal):  2 usages
    def se_deplacer(self):
        print(f"Le poulet marche sur {self.nombre_de_pieds} pattes.")

    def afficher_info(self):  2 usages (2 dynamic)
        print("=== Poulet ===")
        super().afficher_info()
```

Figure 4:ajoute class Chien, Chat, Poulet et Vache.

Cette photo représente en ajoutez les classes filles : Chien, Chat, Poulet et Vache qui héritent de la classe Animal et implémentez la méthode Se_deplacer() dans les classes filles et redéfinissez la méthode Afficher_info() dans les classes filles pour afficher les caractéristiques.

Question 8_16:

```python
class Zoo:  1 usage

    def chercher_animal(self, espece):  1 usage
        for a in self.animaux:
            if a.espece.lower() == espece.lower():
                a.afficher_info()
                print("------------------")

    def nombre_animal(self, nom_scientifique):  1 usage
        return sum(1 for a in self.animaux if a.nom_scientifique == nom_scientifique)

    def tri_animal(self):  1 usage
        self.animaux.sort(key=lambda x: x.code)

    def afficher_animaux(self):  2 usages
        for a in self.animaux:
            a.afficher_info()
            print("------------------")

    def enregistrer_animaux(self):  1 usage
        with open("animals.txt", "w") as f:
            for a in self.animaux:
                f.write(f"{a.code},{a.nom_scientifique},{a.espece},{a.nombre_de_pieds},{a.pays_origine}\n")

    def load_json(self, fichier):
        with open(fichier, "r") as f:
            data = json.load(f)
        for item in data:
            try:
```

Figure 5: Écrire les méthodes.

Cette photo représente en ajoutez une méthode Ajouter_Animal() et Supprimer_Animal() par code , écrire une méthode Update_Animal() par code et Chercher_Animal() par code Nombre_Animal() et Tri_Animal() par code et Afficher_Animal() et Enregistrer_Animal().

Question 17:

```python
class Zoo:  1 usage
    def load_json(self, fichier):
                espece = item["espèce"]
                if espece == "Chien":
                    animal = Chien(**self._map(item))
                elif espece == "Chat":
                    animal = Chat(**self._map(item))
                elif espece == "Poulet":
                    animal = Poulet(**self._map(item))
                elif espece == "Vache":
                    animal = Vache(**self._map(item))
                else:
                    animal = Animal(**self._map(item))  # fallback
                self.ajouter_animal(animal)
            except:
                pass

    def _map(self, item):  5 usages
        return {
            "code": item["code"],
            "nom_scientifique": item["nom_scientifique"],
            "espece": item["espèce"],
            "nombre_de_pieds": item["nombre_de_pieds"],
            "pays_origine": item["pays_d_origine"]
        }
```

```python
if __name__ == "__main__":
    zoo = Zoo( nom: "Zoo Tamesna",  superficie: 5000)
    a1 = Chien( code: 1,  nom_scientifique: "Canis lupus familiaris",  espece: "Chien",  nombre_de_pieds: 4,  pays_origine: "Global")
    a2 = Chat( code: 2,  nom_scientifique: "Felis catus",  espece: "Chat",  nombre_de_pieds: 4,  pays_origine: "Egypte")
    a3 = Poulet( code: 3,  nom_scientifique: "Gallus gallus domesticus",  espece: "Poulet",  nombre_de_pieds: 2,  pays_origine: "Inde")
    zoo.ajouter_animal(a1)
    zoo.ajouter_animal(a2)
    zoo.ajouter_animal(a3)
    print("=== Tous les animaux ===")
    zoo.afficher_animaux()
    print("=== Recherche Chat ===")
    zoo.chercher_animal("Chat")
    print("=== Tri ===")
    zoo.tri_animal()
    zoo.afficher_animaux()
    print("=== Nombre ===")
    print(zoo.nombre_animal("Felis catus"))
    zoo.enregistrer_animaux()
```

Figure 6 et 7: une méthode Load_JSON().

Cette photo représente en écrire une méthode Load_JSON() permettant de lire un fichier Json en local.

# Conclusion:

Ce projet nous a permis de créer un système de gestion d'un zoo en utilisant la programmation orientée objet en Python. Nous avons appris à : Créer des classes et des objets (Animal, Zoo) Ajouter, supprimer, modifier et rechercher des animaux Trier et afficher les informations Sauvegarder les données dans un fichier pour une utilisation future Ainsi, ce programme fournit une gestion complète et organisée des animaux dans un zoo.